MATHEMATICAL PROOF-LANGUAGE DESIGN

# From Proof Obligations to Checkable Plans

A critical synthesis of the nine round-4 reports

---

**Central recommendation.** Build one Lean authoring path that connects a fixed mathematical request, a bounded rule registry, explicit alternative premises, and a replayable proof of the original statement. Combine that path with a small behavioral abstraction whose admissible observations are proved.

This round provides executable components for that design. Their strongest convergence is a calculus of conditional proofs; their strongest differences concern rule grounding, explanation, coverage, and mathematical interpretation. Fresh reproduction and adversarial probes reveal why accepting a proof plan, emitting Lean, and completing the author's request require separate checks.

Prepared for Vladimir Reshetnikov

Codex | 6 September 2026

Self-contained: no earlier report is required. Input snapshot: `58ced1ce667b3ba1ed162a0c6280959536c8bc5f`. Nine packages, 159 input artifacts, 325 supplied PDF pages. Fresh Python reproduction and serial Lean checks are reported with their precise scope; no controlled productivity study or complete Lean frontend is claimed.

**Abstract**

A proof language becomes more natural when it reconstructs the routine justification of a mathematical move while keeping the move's meaning visible. Alder, Bryony, Clover, Fennel, Heather, Juniper, Laurel, Rowan, and Sorrel agree on properties of fixed objects, explicit construction choices, relational contracts, bounded inference, and ordinary Lean evidence. Round four makes part of this agreement operational. Six companions compute antichains of sufficient premise sets; Clover grounds concrete endomorphism rules; Heather connects a restricted list language to realizable observations; Rowan explores finite grounding and semilinear observation images.

This synthesis develops a common semantics, proves the finite support calculus's guarantees on paper, reconciles affine frames with semilinear images, and evaluates all nine implementations. It reproduces their Python experiments, attempts every supplied Lean file, and retains counterexamples to export hygiene, request-relative route selection, and overinterpretation of explanatory frontiers. It distinguishes conditional implication templates from mathematical registrations, partial search from globally minimal results, and valid theorems from faithful completion of a request. The recommended next iteration is a shared Lean implementation and matched authoring experiment, with explicit stopping conditions for a separate language or kernel extension.

# Contents

# 1 The decision this round makes possible

The useful next artifact is a working connection between a mathematical use site and its proof obligations. For example, an author should be able to use an exact integer quotient in a rational calculation, see that divisibility and denominator nonzeroness are the relevant requirements, choose among justified ways to establish them, and obtain a proof of the requested identity. The interface should preserve which quotient, which field, which hypotheses, and which conclusion were intended.

Lean already has dependent types, structures, implicit and automatic parameters, extensible elaboration, and tactics. The proposed work builds on these mechanisms. Its potential contribution is a systematic interface for making existing evidence available at use sites, explaining missing evidence, preserving choices, and presenting the resulting argument. Ordinary Lean term elaboration and kernel checking remain the final logical boundary [12, 13].

The current reports improve on an exclusively architectural discussion: their finite reference implementations can actually parse small documents, propagate requirements, check derivations, and sometimes export proofs about concrete objects. That is meaningful progress. It does not yet establish a general mathematical elaborator or a gain for mathematicians using it. Our recommendation is to combine the complementary components, then measure the complete path with the same libraries and automation available in the Lean control.

## 1.1 What is common, and what the agreement means

All nine reports endorse the following design commitments, with different levels of implementation: preserve a fixed object's identity while learning properties; distinguish data selection from proof search; expose residual obligations; keep inference bounded; preserve scope and observation-specific relations; and compare with an equally equipped Lean baseline. Their agreement is architectural convergence. Each report read the same earlier syntheses and related repositories; the nine documents therefore do not constitute independent usability experiments.

The more informative implementation split is this:

- **Six support-family engines:** Alder, Bryony, Fennel, Juniper, Laurel, and Sorrel implement finite propositional alternatives, with sufficient premises and replayable symbolic derivations.
- **One concrete grounding engine:** Clover works with endomorphism terms and a small semantic rule library, using first-proof closure rather than a complete minimal-support inventory.
- **One concrete behavioral frontend:** Heather interprets a list grammar over restricted observation domains and exports concrete Lean goals; its small requirement resolver has a different role from an antichain solver.
- **One complementary finite model:** Rowan combines typed term generation, first-proof closure, explanatory leaves, and supplied semilinear observation images. Its explanatory frontier is not a sufficient-support antichain.

These are components to evaluate separately, not nine interchangeable implementations of the same language.

### 1.2 Evidence and provenance

The branch was fast-forwarded to fetched `main` at `58ced1c` before this review. The input register records all 159 round-four Git blobs and checkout hashes. It also pins both revised round-three synthesis source/PDF pairs as secondary context. The new `main` reconciles those syntheses; their corrections are incorporated below rather than treated as additional independent proposals. Earlier execution receipts retain their historical meaning and are not silently refreshed to describe new bytes.

We distinguish five kinds of evidence throughout: a design proposal; a written mathematical argument; static source inspection; a fresh executable result; and a controlled authoring measurement. A successful Python checker establishes behavior of that execution. A successful Lean check establishes acceptance of that exact source under the recorded compiler and dependencies. Neither alone establishes a correct source-to-Lean compiler. Supplied PDF page counts are metadata; this review rebuilds and visually checks its own PDF, not all nine input PDFs. Reproduction details appear in Section 9.

## 2 A common semantics for mathematical authoring

### 2.1 Objects, evidence, and unresolved work

Let $\Gamma$ be an ordinary Lean context. An object has a carrier type, $\Gamma \vdash t : A$. A proved view of that same object is evidence $\Gamma \vdash p : P(t)$. Learning injectivity of $f$ does not replace $f$; learning positivity of $x$ does not choose a different real number. A local property index can make these proofs discoverable without changing the carrier's representation. At a module boundary, a structure or subtype may package the object and its laws when that is the useful API.

A construction is different. Choosing a basis, a root, an inverse, or a representing algebra creates or selects data. Those choices can affect later terms and dependent types. Existence of a witness is not automatically an executable construction of one. In particular, proof irrelevance does not license conflating distinct roots, algorithms, measures, or algebra structures.

An unfinished step has a conditional meaning. Write

$$\Gamma \vdash e \rightsquigarrow (t : T; \Delta), \qquad \Delta = (h_1 : H_1, \ldots, h_k : H_k(h_1, \ldots, h_{k-1})).$$

The elaboration promises a term of $T$ when the ordered telescope $\Delta$ is supplied. The hypotheses are not facts in $\Gamma$. For example,

$$(x : A,\ h : P(x),\ q : Q(x, h))$$

cannot be flattened into a set of untyped strings. Composition of unfinished steps substitutes earlier terms and evidence into later obligations and preserves the resulting order. Alder, Bryony, Juniper, and Sorrel make this conditional interface particularly explicit [1, 2, 6, 9].

At the source level, four actions should remain distinguishable:

| Action | Logical effect | Required visible result |
|---|---|---|
| Introduce | Fix an object or declared assumption. | Its type, scope, and mathematical choices. |
| Prove | Establish the exact requested proposition. | A closed proof in the existing context. |
| Explain | Describe sufficient additional premises. | Conditional routes; no context strengthening. |
| Construct | Select a witness satisfying a specification. | The selected data and evidence about those data. |

The same interface may expose these actions in prose, structured commands, or an editor. The semantic distinction matters more than their spelling.

### 2.2 Behavioral contracts preserve the actual intermediate

For a fixed function $f : A \to B$, let

$$\mathsf{Beh}(P, Q, f) \;:=\; \forall x : A,\; P(x) \to Q(x, f(x)).$$

Here $Q$ can relate an input to its output. It can state length preservation, an error bound, or an algebraic law. Properties of an entire function, such as injectivity or continuity, may quantify over multiple evaluations and are also ordinary propositions about the fixed $f$.

If $\mathsf{Beh}(P, Q, f)$, $P' \Rightarrow P$, and $Q \Rightarrow Q'$, then $\mathsf{Beh}(P', Q', f)$. Thus the premise is strengthened and the guarantee weakened. For composition, suppose $f$ guarantees $R(x, f(x))$, $g$ requires $S(y)$, and $R(x, y) \Rightarrow S(y)$. The composite guarantee has the form

$$R(x, f(x)) \;\wedge\; Q(f(x), g(f(x))).$$

The intermediate is the actual $f(x)$, not a newly chosen value satisfying $R$. This detail prevents an attractive relational explanation from proving the behavior of a different program. A requirement transformer that computes sufficient input conditions is useful; it becomes a *weakest* precondition only with a separate minimality or equivalence theorem.

### 2.3 A fixed request may still contain legitimate parameters

Before proof search, freeze the target's meaning-bearing data: its elaborated type, universe levels, structure arguments, indices, measures, regions, and chosen witnesses. Parameterization is allowed; the target need not be a closed term with no variables. What must be prevented is solving a meaning-bearing hole by choosing an easier statement during a search that was advertised as proving the original one.

For example, a request concerning a fixed $n$ in $\mathrm{Fin}(n)$ differs from permission to synthesize $n$. A statement over a fixed measure $\mu$ cannot be repaired by silently choosing the zero measure. A source-level object with an unresolved algebra instance should be resolved or returned as an explicit choice before the proof service is invoked. Rowan's sealed requests and the other reports' frozen semantic headers differ in enforcement. Rowan's strict seal disallows unresolved metavariables in the target and local declaration types and values. Laurel and Sorrel classify holes by role, allowing proof holes and explicitly authorized witness construction while rejecting unresolved meaning choices. Both can preserve fixed free variables; both can construct a witness for a fixed existential specification. The strict seal is a useful first implementation, while the classified policy may admit more staged elaboration at the cost of a more demanding invariant [8, 7, 9, 5].

### 2.4 Truth, statement fidelity, and method fidelity

Three acceptance checks answer different questions:

1. **Truth:** does the generated proof have its claimed Lean type?
2. **Statement fidelity:** is that type the author's intended target, in the intended context, with the intended residual status?
3. **Method fidelity:** does the checked argument respect an explicitly required method or support restriction?

A conditional theorem can pass the first check while failing to complete an unconditional request. A proof can cite a named rule redundantly and satisfy a syntactic method policy without making that rule indispensable. Dependencies should therefore be recorded as declared prerequisites, supplied evidence, and evidence actually used. None of those records alone proves necessity.

## 3 The shared finite calculus of sufficient premises

### 3.1 The exact finite problem

Fix a finite set $U$ of ground propositions, a finite set $R$ of Horn rules

$$a_1 \wedge \cdots \wedge a_k \Longrightarrow b,$$

a set $K \subseteq U$ of currently established facts, and a finite set $O \subseteq U$ of premises the author permits the explanation service to offer. Each mathematical registration must eventually carry a Lean proof of its implication. In the propositional reference engines, the implication is instead an explicit theorem parameter. That substitution tests the support calculus without proving the intended mathematical interpretation of an atom.

For a target $q$, the desired answer is

$$\mathsf{Req}_K(q) = \mathsf{Min}_{\subseteq}\{S \subseteq O : q \in \mathsf{Cl}_R(K \cup S)\}. \tag{1}$$

The result is an antichain: no retained support strictly contains another. Minimal means inclusion-minimal in the stated finite profile. It does not mean fewest symbols, least cognitive cost, logically weakest among arbitrary Lean propositions, or mathematically necessary.

Two different kinds of empty result have opposite meanings:

$$\begin{aligned} \mathsf{Req}_K(q) = \{\varnothing\} &\quad \text{means established,} \\ \mathsf{Req}_K(q) = \varnothing &\quad \text{means no offered route in the complete profile.} \end{aligned}$$

The latter does not prove $\neg q$. An interrupted run cannot even assert that no route exists within the full profile.

### 3.2 Propagation with proof templates

Initialize each $a \in K$ with the empty support and its current proof; each $a \in O$ is also offered the singleton support $\{a\}$ and a formal premise variable, subject to immediate domination if an empty

support already exists. For a rule with premises $a_1, \ldots, a_k$, choose one support $S_i$ for each premise, form $S = \bigcup_i S_i$, and apply the rule proof to their proof templates. Insert $(S, p)$ at the conclusion unless an existing support is a subset of $S$; remove any existing strict supersets. A rule with no premises has the empty union and needs its own genuine nullary evidence.

The algebra is compact:

$$\mathcal{A} \oplus \mathcal{B} = \mathsf{Min}(\mathcal{A} \cup \mathcal{B}), \qquad \mathcal{A} \otimes \mathcal{B} = \mathsf{Min}\{A \cup B : A \in \mathcal{A}, B \in \mathcal{B}\}.$$

Alternative derivations combine with $\oplus$; conjunctive requirements combine with $\otimes$. This resembles established provenance algebras, in which alternatives and joint dependencies have separate operations. It should not be presented as an entirely new inference principle. The proposed increment is its use with mathematical source requests, conditional proof interfaces, scope, methods, and checked Lean instantiations [14, 2].

**Theorem 3.1** (Finite-profile guarantees)**.** *With fixed finite $U, R, K, O$, fair saturation computes exactly* (1)*. Every retained support has a conditional derivation of its target. The completed support families are independent of rule schedule, although selected proof representatives can differ.*

*Proof.* Soundness follows by induction on inserted proof templates. Initial templates are facts or explicit assumptions; a rule application combines valid templates under the union of their assumptions. Removing a superset is safe because the retained subset's proof works whenever that superset is supplied.

For termination, interpret a family $\mathcal{A}$ by its upward closure $\uparrow \mathcal{A} = \{S \subseteq O : \exists A \in \mathcal{A}, A \subseteq S\}$. Each effective insertion strictly enlarges this finite set for some atom. Removed supports remain dominated, so cannot reappear as effective insertions. There are at most $|U|2^{|O|}$ atom/support pairs to discover.

For completeness, any derivation from $K \cup S$ is a finite Horn proof tree. Induct on that tree. Initialization covers its leaves. Fair saturation eventually combines supports contained in $S$ for every rule's premises, yielding a support contained in $S$ at its conclusion. If $S$ is minimal, the resulting support must equal $S$. Soundness gives the converse and uniqueness of the final minimal family. □

This is a written proof of the specified calculus. The finite tests in the companions are additional implementation evidence; they do not formalize this theorem about their Python source.

### 3.3 What partial search actually preserves

The upward-closure order is also the correct way to describe progress. A partial family need not be a literal subset of the final minimal family. Suppose a first discovered proof of $q$ requires $B$, and a later proof requires nothing. The displayed family changes from $\{\{B\}\}$ to $\{\varnothing\}$. The earlier conditional proof remains sound, while $\{B\}$ is absent from the final minimal family.

This corrects Sorrel's statement that a bounded run retains a subset of the final label families (its source line 845). Laurel explicitly makes the appropriate distinction. The fresh paired probe reproduces a partial $\{B\}$ route followed by a completed empty-support route. A budgeted result should say *sound discovered routes; inventory incomplete; global minimality not certified*. It should not imply that later search merely appends additional members to the displayed antichain [7, 9].

There are at least three different completeness claims: saturation of the given ground graph; coverage of all intended ground instances from a bounded term pool; and completeness for a mathematical class of problems. The first does not imply the second, and neither normally implies the third.

### 3.4 An independent certificate for coverage

Replaying one derivation establishes sufficiency, not completeness of an answer list. Juniper usefully separates derivation validation from a coverage check over the finite profile [6]. A general checker can validate a final family by checking sound templates, the required initial labels, and closure under every rule/support combination modulo domination. By the proof-tree argument above, any derivable support is then covered by some listed subset. Antichain normalization gives minimality.

This does not make completeness checking cheap: a wide rule can expose the same large Cartesian products as inference. It does make the certificate's claim explicit and allows a simpler checker to disagree with a heuristic planner. The request must bind the exact $U, R, K, O$ and policy; a closure certificate for a different registry establishes the wrong coverage fact.

### 3.5 Total supports, missing supports, and author choices

An implementation may first choose a total assumption universe $H = K \cup O$ and compute $\mathcal{S}(q) = \mathsf{Min}\{S \subseteq H : q \in \mathsf{Cl}_R(S)\}$. It then derives missing-premise alternatives by

$$\mathsf{Req}_K(q) = \mathsf{Min}\{S \setminus K : S \in \mathcal{S}(q)\}.$$

The proof object must still remember the total support and how its currently available members are discharged. Choosing the smallest total support is not the same as choosing the easiest remaining work.

This formulation includes the known facts in the total assumption universe. Computing total supports over $O$ alone and then subtracting $K$ can lose proofs that use known facts outside $O$. It also describes a proposition-level common fragment: Fennel can retain distinct named proof roots for the same proposition, whereas Juniper offers propositions directly. A richer formal model uses a finite root set with a map from roots to propositions; it must not collapse different author actions merely because their proposition agrees.

The fresh Sorrel probe makes this concrete. There are two rules, $A \Rightarrow G$ and $B \wedge C \Rightarrow G$, and $B, C$ are available. The model correctly reports an empty residual. Its CLI selects the smaller total support $\{A\}$ for export, yielding a valid but unnecessary conditional theorem. It has proved a route while failing to select the already completed route. Selection should minimize or rank *residual* work first, retain the corresponding complete derivation, and make any remaining author choice explicit. A proposal to add premises to a theorem is never automatic repair of the original theorem.

### 3.6 Methods must constrain search before information is erased

Suppose $P$ follows directly from $A$, and another proof uses $A, B$ and a required theorem $r$. Unrestricted support minimization can delete the second route before a later method filter asks for $r$. A support set alone also forgets which of two proofs with the same premises used $r$.

Filter forbidden rules before saturation. For positive method requirements, retain method-sensitive proof states, such as a pair of support and method automaton state, or run separate profiles. A mandatory root rule, as in Clover, is an executable and understandable policy. The adversarial probe also shows its limit: applying injectivity transport through reflexive equality can satisfy a root-rule requirement when the desired injectivity was already known. That is rule occurrence, not method indispensability.

# 4 Grounding is a different algorithmic problem

The six propositional engines assume that ground atoms and rule instances are already available. A mathematical frontend must produce them from elaborated expressions. This is where representation choices, universes, dependent arguments, and term explosion return.

## 4.1 A bounded protocol with a visible boundary

One conservative protocol first forms a typed pool of subterms from the target and context, optionally expands it by a fixed set of constructors to a stated depth, and then instantiates registered schemas from that pool. For a schema with $k$ independent argument positions and a pool of size $N$, $N^k$ is a crude candidate bound before type and relevance filtering. A finite closure theorem says nothing about the cost of producing this pool. Terms created by witness synthesis must pass through a separate construction boundary and create a new explicit inference problem.

Backwards relevance filtering can reduce the graph before forward closure. Clover goes further than the opaque-atom engines by implementing a restricted endomorphism syntax, collecting terms, grounding a small collection of injectivity and equality rules, and exporting applications of concrete Lean lemmas. Rowan also treats typed generation as part of the finite problem. These contributions should be combined with support inference rather than judged by whether they compute the same antichains [3, 8].

## 4.2 Two separate exponential pressures

Even after grounding is complete, the answer itself can be exponentially wide. If each of $n$ independent requirements can be met by either $a_i$ or $b_i$, a goal requiring all $n$ has $2^n$ incomparable sufficient supports. Fennel's fresh stress run exposes 1,024 routes for ten pairs. Its cap of 128 stored nodes retains 108 target routes and marks the run incomplete. Laurel supplies a separate width experiment. These are demonstrations of output growth, not comparative performance benchmarks [4, 7].

The user interface should therefore support one proved route, a few ranked alternatives, and an optional complete inventory. It must not display a truncated list as exhaustive. Shared proof DAGs can reduce repeated proof construction, and symbolic decision structures may compact some support families, but no representation removes every large output or makes the choice among alternatives free.

## 4.3 Evidence identity under edits and rewriting

An index entry should refer to a typed expression and the context in which its evidence was checked. A printed name, hash, or stale local identifier does not transport a proof into a new context. A first implementation should rebuild the local index after edits; later reuse can require a checked context embedding that preserves dependencies, universes, and instance arguments.

Rewriting also needs a relation-specific proof. Definitional equality permits conversion. An equality proof can transport a predicate through substitution. An equivalence of propositions can transport their evidence in the chosen direction. Neighborhood equality, almost-everywhere equality, and agreement to finite precision support only their registered consumers. The cache should not turn any of these relations into unrestricted equality.

### 4.4 Explanation is not automatically a residual contract

Rowan's leaf frontier is deliberately explanatory and nonminimal. A fresh example with $A \Rightarrow A$ and $A \wedge B \Rightarrow G$ reports $B$ as the leaf frontier from an empty context. Supplying $B$ leaves the goal open, because the seedless cycle has not established $A$. This does not create a false derived proof. It shows why an explanation of a blocked proof search must not be displayed as a sufficient set of assumptions.

Use distinct result types for a diagnostic trace, a checked sufficient route, and a complete minimal-family certificate. The report's phrase "valid nonminimal explanation" should be read in its stated diagnostic sense. The appropriate improvement is to show the unresolved cycle explicitly or offer a separately checked support calculation, not to claim Rowan's proof checker accepted the goal [8].

## 5 Behavioral synthesis on realizable observations

This is the strongest mathematical convergence beyond the support calculus. The reports repair a common error in abstract synthesis: treating every abstract input as if it were a possible source input.

### 5.1 Four contracts behind a finite check

Let $X$ be the source input type, $D : X \to \mathsf{Prop}$ its admissibility predicate, and $\mathsf{obs} : X \to V$ an observation. A fixed source candidate $c$ has an abstract interpretation $F_c$ only when a denotation theorem connects its actual behavior to $F_c(\mathsf{obs}(x))$ for admissible $x$.

Four claims must be separated:

1. **Positive soundness:** successful abstract checking proves the source property, using the denotation bridge and coverage of source inputs.
2. **Decision completeness:** every true source property in the declared fragment passes the abstract criterion.
3. **Negative realization:** a particular failing abstract observation has an admissible concrete input. Together with a theorem that the source contract would imply the violated abstract property, it refutes the fixed candidate; realization alone supplies no such implication.
4. **Observation coverage:** all relevant source inputs are accounted for by the domain theorem used in the positive direction.

Exact characterization of an observation image can establish several of these claims, but it is more than every equality checker needs. Agreement on several tests has no universal force for an arbitrary black-box program; the grammar or exact candidate needs its own denotation theorem.

### 5.2 A realizable affine frame

The following formulation follows Alder's integer-frame version, which carefully avoids a hidden torsion-free assumption [1]. Let $\mathsf{obs} : X \to \mathbb{Z}^d$. Choose admissible inputs $x_0, \ldots, x_m$, put $b = \mathsf{obs}(x_0)$ and $v_i = \mathsf{obs}(x_i) - b$, and prove

$$\forall x,\ D(x) \Longrightarrow \exists k_1, \ldots, k_m \in \mathbb{Z}, \quad \mathsf{obs}(x) = b + \sum_i k_i v_i. \tag{2}$$

No claim is made that every integer combination is realizable.

**Theorem 5.1** (Equality from a realizable frame)**.** *Let $G$ be an abelian group, and let $L, R : \mathbb{Z}^d \to G$ each be a constant plus an additive homomorphism. Under (2),*

$$(\forall x,\ D(x) \to L(\mathsf{obs}(x)) = R(\mathsf{obs}(x))) \quad \iff \quad (\forall i = 0, \dots, m,\ L(\mathsf{obs}(x_i)) = R(\mathsf{obs}(x_i))).$$

*A failed frame test already has an admissible source input. With decidable equality in $G$, the equivalence yields a finite decision procedure.*

*Proof.* The forward implication specializes the universal statement. Conversely, write $H = L - R = c + \ell$, with $\ell$ additive. Agreement at $x_0$ gives $H(b) = 0$. Agreement at $x_i$ gives $\ell(v_i) = 0$ by subtracting $H(b)$. Equation (2) and additivity, including integer multiples, then give $H(\mathsf{obs}(x)) = 0$ for every admissible $x$. □

The generic theorem is a written mathematical result here. The concrete source files compiled in this review are separately enumerated; their acceptance should not be described as a formalization of this whole theorem.

### 5.3 How semilinear images fit the same picture

Rowan supplies an exact image as a finite union of linear sets,

$$S = \bigcup_j \{b_j + \sum_k n_k v_{j,k} : n_k \in \mathbb{N}\} \subseteq \mathbb{N}^d.$$

For integer affine forms with difference $H(z) = c + a \cdot z$, equality on $S$ is equivalent to $H(b_j) = 0$ and $a \cdot v_{j,k} = 0$ for every component and generator. If a base fails, it is a witness. Otherwise a failing direction gives the witness $b_j + v_{j,k}$. To refute the source candidate, attach a source realizer to that point [8].

Each component therefore supplies a finite set of potential distinguishing observations: its base and its one-generator extensions. Alder's frame avoids computing the whole image when realizable test points already span everything needed for affine equality. Rowan's exact-image interface retains more information and naturally represents finite alternatives. Heather makes the frame route concrete for a small list grammar. These are compatible levels of domain information, not competing notions of soundness.

Laurel states the same idea using actual witnessed lifted vectors $(1, \mathsf{obs}(x))$ and rational linear-span coverage. Rowan's bases and base-plus-generator witnesses instantiate that interface. Over a rational affine domain in $\mathbb{Q}^d$, redundant witnessed tests can be reduced to at most $d + 1$ independent lifted vectors while preserving a coverage proof and their realizers. This is a proposed combination of the reports, not an implemented image-discovery algorithm or a replacement for Alder's more general integer/abelian-group hypotheses [7, 8].

For vector spaces over a field, affine-span rank gives a useful test-count criterion. An unrestricted affine scalar form on an $r$-dimensional affine domain requires $r + 1$ affinely independent point evaluations in the worst case to certify that it is identically zero. A single nonzero evaluation already refutes that claim. If the test evaluations do not span that domain, a nonzero affine form can vanish on all tests. This lower bound is relative to the full affine hypothesis class; a restricted source grammar may need fewer tests. Heather explicitly develops the rank perspective [5].

### 5.4 A small table of domains that must give different answers

| Source domain | Realizable probes or image | Correct affine equality criterion |
|---|---|---|
| Independent `List Nat` inputs | $0, e_1, \ldots, e_d$ | All constants and coefficients agree. |
| `List Empty` inputs | Only 0 | Only constant terms matter. |
| One list and its reversal | $(0,0), (1,1)$ span the diagonal | For $c + an_1 + bn_2$, require $c = 0$, $a + b = 0$. |
| One even list length | Image $\{2n : n \in \mathbb{N}\}$; probes $0, 2$ | Integer affine forms still require zero constant and slope. |
| No admissible input | Empty image, with a proof of emptiness | The universal contract is vacuous. |

For `List Empty`, the programs returning the empty list and returning the input have the same length on every actual input, although their raw length expressions are 0 and $n$. For correlated lengths, the abstract pair $(1, 0)$ cannot refute equality between the length of a list and its reversal. An empty collection of tests is not itself a proof that $D$ is empty.

Coefficient domains matter. In a group with torsion, equality at integer multiples does not justify division by those multiples. A rational-span argument and an integer-span argument have different hypotheses. Likewise, inequality does not extend through arbitrary signed affine combinations; it needs an order-compatible coverage argument, such as convex combinations or monotonicity. A language should expose these as distinct proof interfaces.

### 5.5 The bridge must describe the exact program

For lists generated from variables, empty lists, cons, append, reverse, and map by a fixed total function, length normalization follows by induction: empty contributes zero, cons adds one, append adds lengths, reverse and map preserve length. The source grammar must itself be well typed. A closed cons operation over `Empty` cannot be admitted merely because its untyped abstract rule says "add one."

This bridge justifies a length property, not sorting, permutation, stability, or running time. Joint outputs need a joint contract when they share inputs or correlations. A partial program sketch is different again: pruning it requires a theorem about all permitted completions, not a counterexample to one arbitrary completion. Leant integration should retain these distinctions even when a fast abstract model is used only to rank candidates.

## 6 Mathematical interfaces worth compressing

The proposals' larger examples should guide the first registrations, but compact illustrative syntax is not evidence that a companion parses or proves those examples. This section gives the relevant mathematics directly.

### 6.1 Local differentiation: preserve the region before differentiating

Suppose $s \subseteq \mathbb{R}$ is open, $x \in s$, $y$ is differentiable at $x$, and $\operatorname{deriv} y(t) = F(y(t))$ holds for every $t \in s$. Openness and membership give a neighborhood of $x$ on which the two functions agree. Their derivatives at $x$ agree, and a chain rule, with its differentiability hypotheses for $F$, gives the desired second-derivative expression.

The type-directed step is not “differentiate any displayed equality.” It consumes neighborhood equality, differentiability of the appropriate functions, and a point in the relevant domain. A regional induction hypothesis must remain available as a quantified scheme $\forall y \in s,\ P(y)$, not just as its already-used instance at $x$. Controlled specialization can then provide neighborhood facts and later instances. Equality only at $x$ is insufficient: the constant zero function and $t \mapsto t - x$ agree at $x$ but have different derivatives there. A good interface displays the region and supplies the routine neighborhood conversion automatically. This is a recurring example in Alder, Clover, Fennel, Heather, and Rowan [1, 3, 4, 5, 8].

More generally, a consumer should declare which relation it respects: global equality, equality near a point, equality almost everywhere under a fixed measure, equality modulo an ideal, or equality of a finite jet. A proof of the consumer’s congruence law is what permits the corresponding compression.

### 6.2 Exact quotients: three operations behind one slash

Integer division, rational division, and inversion in an algebra are different operations. If $d \mid n$ in $\mathbb{Z}$, integer division has the exact balance law $d(n/d) = n$. To transport it into a field $K$, prove that the image of $d$ in $K$ is nonzero, then cancel to obtain

$$\iota(n/d) = \iota(n)/\iota(d).$$

The first premise concerns exactness in the source ring; the second concerns invertibility in the target. Integer nonzeroness alone is not enough for a field of positive characteristic. In $\mathbb{Q}$, a nonzero integer has nonzero image.

The inherited Frey coefficient benchmark makes the distinction operational. For integers $a, b$ and an exponent $p \in \mathbb{N}$, keep the satisfiable context

$$2 \mid b, \quad 4 \le p, \quad p \text{ odd}, \quad a \equiv 3 \pmod 4.$$

The fixture is $(a, b, p) = (3, 2, 5)$: the third coordinate is the exponent. No Fermat equation, primality, nonzeroness, or coprimality is required for these coefficient facts. Their nonvacuity should be established directly, rather than by importing an inconsistent hypothetical Fermat counterexample.

For the fourth coefficient’s divisibility, evenness of $b$ and $p \ge 4$ already imply $16 \mid b^p$, hence $16 \mid a^p b^p$. This route does not require oddness of $p$ or the residue of $a$. For a second coefficient involving the numerator $b^p - 1 - a^p$, the residue and oddness give $a^p \equiv -1 \pmod 4$ and $b^p \equiv 0 \pmod 4$, so the numerator is divisible by four. Juniper sharpens the sufficient lower bound for this consumer to $p \ge 2$: evenness already makes $4 \mid b^p$ at that bound. The fixture $(a, b, p) = (3, 2, 3)$ satisfies this route, while its fourth-coefficient numerator need not be divisible by sixteen. This is a useful library-premise improvement, which antichain minimization cannot discover if the stronger old rule is the only registered rule. The exact formula and cast target belong in the request header [6].

The finite engines’ atoms named `DivSixteen`, `CastCorrect`, or `coefficient_identity` do not themselves formalize this arithmetic. Their Lean exports quantify over arbitrary propositions and the implication rules connecting them. They establish conditional assembly. A production registration must connect these rules to the actual divisibility and cast lemmas. Heather’s use-site theorem proves readiness premises $d \neq 0 \wedge d \mid n$ from positivity and divisibility; it does not implement an exact quotient. These distinctions are central to a fair evaluation.

### 6.3 A demanded representation can introduce new obligations

Fennel's finite product $P_n(x) = \prod_{j<n}(1 - xq^j)$ has an unconditional polynomial derivative obtained by the product rule. The convenient quotient presentation

$$P_n'(a) = -P_n(a) \sum_{j<n} \frac{q^j}{1 - aq^j}$$

requires $1 - aq^j \neq 0$ for each factor. At $n = 1$ and $a = 1$ over $\mathbb{R}$, the true derivative is $-1$, while the unguarded right side under totalized zero division is 0. At $n = 0$, the empty product and empty sum need no guard. A property-aware language should preserve the unconditional polynomial representation and expose additional demands only when the quotient form is requested. This is a concrete use for representation-sensitive requirements, not a failure of Lean to express the derivative [4].

### 6.4 Almost-everywhere reasoning has quantifier and measure indices

An integral congruence theorem can use equality almost everywhere with respect to the exact measure of the integral. It does not authorize pointwise evaluation at an exceptional point. When a library defines a totalized integral, congruence and interchange can also have different hypothesis requirements: adding an integrability guard to every use of integral congruence can overstate the API, while omitting the hypotheses of a Fubini route can make interchange invalid.

Sorrel's concrete example can be stated without probability normalization. Let $\mu$ be a finite positive measure on $\mathbb{R}$, supported almost everywhere in $[a, b]$ with $a \leq b$, and let $z \in \mathbb{C}$ lie outside that interval. Set $M = \mu(\mathbb{R})$ and $F_\mu(t) = \mu((-\infty, t])$. Then

$$\int_{\mathbb{R}} \frac{d\mu(x)}{z - x} = \frac{M}{z - b} - \int_a^b \frac{F_\mu(t)}{(z - t)^2}\, dt.$$

To see the identity, write $F_\mu$ as an integral of $\mathbf{1}_{x \leq t}$, interchange the integrals using compactness, finite mass, and the nonzero denominator, then use $\int_x^b (z - t)^{-2}\, dt = (z - b)^{-1} - (z - x)^{-1}$. The outer $dt$ is Lebesgue measure. Strict and non-strict CDF conventions can differ at atoms of $\mu$ yet give the same $dt$ integral; they do not justify deleting those atoms from $d\mu$. The formula includes mass-two measures and the degenerate interval $a = b$, both useful controls [9].

Bryony's weighted-subgraph example emphasizes the measure and fibers; Sorrel's Cauchy example emphasizes retaining atomic contributions. A strict versus non-strict fiber condition can be invisible under a nonatomic measure and visible under a point mass. Such paired examples are better acceptance tests than replacing every equality relation by a generic "almost equal" annotation [2, 9].

Quantifier order is another essential guard. For a countable index set, $\forall n$, a.e. $x$, $P(n, x)$ can often be combined into one almost-everywhere statement for all $n$, using a countable union of null exceptions. The uncountable version fails: under Lebesgue measure, for each $t \in \mathbb{R}$ we have $x \neq t$ almost everywhere, but no $x$ differs from every real $t$. Clover and Bryony explicitly preserve this boundary. The language should register the countable interchange theorem with its index hypothesis. A conditional positive bridge is also useful: for continuous real-valued functions and a measure positive on every nonempty open set, almost-everywhere equality implies equality everywhere. Otherwise a point of inequality would have a nonempty open neighborhood of inequality and hence positive measure. The additional continuity and measure hypotheses are exactly what authorizes that consumer.

### 6.5 Definitions should export laws of the construction

A basis, quotient, representing object, or tensor family should expose its universal property and the naturality or computation laws needed by consumers. For a representing algebra built from a finite family of factors, finiteness of the *index set* is not finiteness of each factor as a module. A singleton family containing $\mathbb{Q}[X]$ is a counterexample to that inference. The earlier synthesis corrected an exposition that omitted finite/free factor hypotheses; the round-four reports appropriately retain them.

This suggests an interface beyond local adjectives: a definition package should export the construction, its canonical maps, the universal property, and any additional finiteness assumptions explicitly. Infer routine uses of those laws, while preserving which witness and structures the author chose. Automating the theorem connecting a representing object to its property is valuable even if the construction itself stays ordinary Lean.

### 6.6 Certified computation should return the relation it establishes

Alder's formal-root example and Juniper's inverse-polynomial example show why a CAS interface needs a mathematical contract. A truncated power series is a jet, not automatically a convergent analytic function. A residual bound for a polynomial equation needs a branch-sensitive theorem to bound error to a selected root. A nonzero derivative at a point is not a uniform lower bound on a neighborhood; the latter is what some error estimates consume.

For a precise inverse-branch contract, let $U, V \subseteq \mathbb{R}$ be open, let $f$ be smooth on $U$, and let the chosen $g$ be continuous on $V$. Assume $g(V) \subseteq U$, $f(g(y)) = y$ on $V$, and $f'(x) \neq 0$ on $U$. Repeated differentiation then gives, for $n \geq 1$ and $y \in V$,

$$g^{(n)}(y) = \frac{P_n(f'(x), \ldots, f^{(n)}(x))}{f'(x)^{2n-1}}, \qquad x = g(y).$$

Writing $u_i = f^{(i)}(x)$ and $\mathcal{D} = \sum_i u_{i+1}\partial/\partial u_i$, the polynomial recurrence is

$$P_1 = 1, \qquad P_{n+1} = u_1 \mathcal{D} P_n - (2n-1)u_2 P_n.$$

It follows by the chain rule and $dx/dy = 1/f'(x)$. Juniper computes such polynomials exactly and checks finite identities. Those executions do not prove the analytical inverse-function hypotheses or a general verified CAS [6, 1].

Continuity of the chosen branch is substantive. For $f(x) = x^2$ on $U = \mathbb{R} \setminus \{0\}$ and $V = (0, \infty)$, choose $g(y) = \sqrt{y}$ at rational $y$ and $g(y) = -\sqrt{y}$ at irrational $y$. The right-inverse law and nonzero $f'$ on $U$ hold, but $g$ is discontinuous. The formal recurrence cannot repair that missing analytical premise.

Similarly, an identity certificate $p = \sum_i q_i f_i$ proves ideal membership after the polynomial denotations are connected to the original expressions. An approximation certificate, a modular identity, and an exact equality have different consumers. Reuse a common request and denotation interface, while allowing domain-specific reifiers and checkers. Do not insert expensive or incomplete algebraic reasoning into definitional equality merely to hide these contracts.

# 7 What each report contributes, and what to retain

The following evaluations concern final artifacts at the input pin. The reading memos and machine-readable convergence records give exact source anchors. Distinctive contributions can be complementary even when several reports independently develop a similar theorem.

## 7.1 Alder: residuals and integer affine frames

Alder provides a particularly coherent explanation of unfinished work as dependent residuals, with publication separate from explanation. Its actual parser enforces scoped assumptions and rejects unsupported claims; its separate derivation checker replays support proofs. The mathematical frame theorem explicitly handles integer combinations, torsion, unrealizable points, and an empty admissible domain. Its formal-root discussion adds a useful precision and branch interface beyond the common quotient examples. For instance, $\Delta + 4\Delta^2 = (4/9)Q$ has the selected zero-constant branch $\Delta = (\sqrt{1 + (64/9)Q} - 1)/8$. The other exact root has zero equation residual as well, but constant term $-1/4$ and therefore violates the selected branch contract. Residual checking alone cannot choose the requested root.

Retain the separation of explanation from context mutation, the integer-frame theorem, and the branch-sensitive certificate path. The two supplied Lean exports are conditional propositional templates; neither certifies that an atom named `derivative_eq` means a derivative theorem. Grounding, dependent Lean integration, and actual source reification remain proposals [1].

## 7.2 Bryony: requirement families with exact consumers

Bryony explains support families as a reusable interface for proof, repair, and synthesis. Its source-level treatment of weighted Fubini, measure-specific consumers, and dependent structure choices is particularly useful. It separates grounding completeness from closure completeness and binds certificates to the full source snapshot and keeps offered premises conditional until proved.

Retain its precise consumer contracts, two-layer completeness distinction, and independent identity/replay checks. Its Frey specimen is a generic implication export. The frontier can expose a direct exactness hypothesis as an alternative to lower-level parity/exponent premises, but that does not make the alternatives globally weakest mathematical assumptions [2].

## 7.3 Clover: the bridge from terms to rules

An actual parsed fragment from Clover's demonstration is:

```
fix f g r k : End
assume retraction : left_inverse r f
assume outer : injective g
claim composite : injective (g . f)
```

The mathematical proof derives injectivity of $f$ from its left inverse, then injectivity of $g \circ f$ from the two injectivity proofs. Its emitted Lean theorem quantifies over the carrier and those same functions and assumptions. This small example demonstrates genuine source-to-rule work, while the richer regional and dependent syntax in the article remains proposed.

Clover is the most direct complement to the six propositional planners. Its finite term language includes endomorphisms, composition, injectivity, left inverses, involutions, and pointwise equality. The supplied Lean core proves seven concrete rules, and the generated examples apply them. A comparison of rule profiles exposes grounding and closure costs that a pre-grounded antichain demo cannot reveal.

In the reproduced profile comparison, a full registry generates 211 instances and performs 633 repeated-scan rule tests; the two-rule profile generates 16 instances and performs 48 tests. Both retain a four-node proof. These are Python model operation counts on one fixture, not a Lean speedup measurement.

Retain the concrete semantic registry, explicit term-generation boundary, scoped proof spine, and measured profile comparison. Do not label first-proof closure as minimal-support inference. The demonstration deliberately leaves an unsupported strengthening unfinished; three exported theorems are not acceptance of the whole document. Fresh adversarial exports also expose name-hygiene failures: parsed identifiers can become Lean keywords or shadow opened registry names. The correct next step is hygienic expression-based emission, fresh declaration names, fully qualified references to semantic predicates and rule constants, and compilation of the exact emitted artifact [3].

### 7.4 Fennel: alternative requirements with visible resource limits

Fennel implements a parser, antichain planner, independent certificate checker, and four conditional Lean exports. Its best contribution is making the exponential frontier visible and reporting incompleteness under a cap. The distinction between total and remaining prerequisites is also useful for the second-coefficient example, where a cached certificate and an arithmetic route require different remaining work.

Retain the stress fixture, resource-status vocabulary, and method-policy separation. Its larger mathematical surface is illustrative; its atoms are opaque, and the generated rules remain explicit implication hypotheses. Its finite tests are extensive enough to investigate the reference algorithm, but they do not settle dependent grounding or human comprehension [4].

### 7.5 Heather: actual observation domains and concrete exports

Heather provides the most useful behavioral implementation slice: a parsed list language, affine interpretation, domain-specific certificates, realizable negative witnesses, and generated Lean statements. Free, empty, diagonal, and even-length domains exercise different admissibility facts. Its rank argument asks how many observations the chosen hypothesis class really needs.

Retain the domain diversity and concrete source targets. The difference between a model checker and an exporter is experimentally visible: an accepted model can emit a tactic sequence that attempts another tactic after an earlier one closed the goal. The review preserves original compiler results and any repair as separate evidence. Its readiness theorem is intentionally weaker than an exact-division implementation, and its domain checker is not a verified compiler for arbitrary list programs [5].

### 7.6 Juniper: separate proof replay from coverage

Juniper has the clearest explicit split between a conditional derivation certificate and a finite coverage certificate. It includes schedule reversal, an exhaustive-subset oracle on generated profiles, and

interrupted runs that do not claim complete inventories. Its exact inverse-polynomial experiment adds a different mathematical service with a crisp denotation boundary.

Retain the coverage checker and explicit attempt budget, together with the local analytical hypotheses required by the inverse recurrence. The budget is a bound on attempted combinations, not a hard wall-clock or total-memory guarantee. Its two generated routes prove generic implications and do not reify topology, arithmetic, or derivatives [6].

### 7.7 Laurel: minimal frontiers and honest partial results

Laurel offers an independently replayed support calculus, alternative exact-quotient routes, a seedless-cycle fixture, and a width experiment. Its careful statement of partial-result minimality should become shared wording. Its sharp-regularity example also insists that a useful interface preserve the strength of a conclusion, rather than compressing a theorem by silently replacing an optimal regularity result with a bound.

Retain the complete-profile versus interrupted-result distinction and the paired cycle tests. All nonempty supplied Lean exports are propositional templates. The cycle export contains no theorem at all; compiler acceptance of that file is useful hygiene evidence but no positive theorem evidence. The next implementation should retain a linked derivation when selecting a remaining-premise alternative [7].

Laurel also implements a useful symbolic repair operation, `reopen_known`. If a retained derivation used a removed fact $h : P$, it replaces that known leaf by a residual variable and abstracts the proof to $P \to G$. Removed facts not used by the derivation need not appear in the new telescope. The implementation requires identical atoms and rules, creates a new context identity, and independently checks the reconstructed route. The result is sufficient but may no longer be minimal. This is a concrete middle ground between discarding every proof after an edit and reusing stale evidence.

### 7.8 Rowan: exact images, bounded generation, and sealed requests

Rowan's semilinear-image criterion gives constructive abstract witnesses and handles empty, correlated, and finitely alternative observation domains. Its finite typed pool and sealed request keep two difficult boundaries in view: generation must actually terminate, and search must not choose the meaning of its target. These are valuable design constraints even though no Lean source is supplied.

Retain those contributions and the source-realization requirement. Its explanatory leaf frontier should remain a diagnostic, as the cycle-plus-leaf probe demonstrates. The model's many concrete list comparisons and affine pair checks support its finite implementation behavior; they do not prove image discovery for arbitrary programs or all-completion sketch pruning [8].

### 7.9 Sorrel: reusable conditional plans, with selection still to fix

Sorrel most directly treats a residual proof interface as an object that can be composed or reused as a mathematical lemma. Its contract-typed plan separates construction choices, conditional proof work, and method records. The Cauchy example usefully emphasizes exact atomic terms and relation-specific consumers, while its support calculus has the same finite mathematical guarantees as the shared model.

Retain the ordered plan interface and the explicit synthesis handoff. Correct the prose claim about partial families and the CLI's total-support selection behavior. The latter is a request-relative usability

and completion problem, not a false theorem accepted by Lean. Its supplied selected plan remains a generic conditional implication [9].

# 8 Leant integration: build on the existing acceptance boundary

The user specifically requested attention to Leant's automatic term synthesis and tactic suggestion. The committed baseline inspected in this campaign is `823259f7e3c6d24e990d3f48f78c4f1c4f88059e`. Timestamped live metadata records a later `2d400ed8178119c922d644b509d504c097e944ba` checkout, which continued changing during the audit. The implementation claims below concern the immutable `823259f` baseline, not an audit of the later revision or its working changes. No Haskell build, synthesis server run, or full Leant test suite was performed for this synthesis.

## 8.1 Existing mechanisms should not be proposed as missing

Leant's candidate verification can combine an exact rendered candidate's type check with a behavioral assertion before accepting it. Assertions checked through Lean can themselves be universal when executable decision procedures support the proposition; it would be inaccurate to describe every behavioral assertion as mere finite testing. The private `Verified` receipt deliberately records acceptance by its supplied callback and retains the exact candidate. Its source documentation explicitly distinguishes that receipt from a solver certificate, behavioral proof object, or kernel proof [15].

The length integration already has exact source-spine identities, argument roles, provider transfers, and joint-output contracts. These are a stronger baseline than independent output-length tags. Their association with a candidate does not, by itself, prove an abstract transfer law about the candidate's Lean implementation. The additional research target is a retained source-denotation bridge, an admissible-domain theorem, and an original-target proof assembled from the successful check.

For tactic suggestions, the relevant boundary is replay of the exact displayed executable text in the intended proof context, with all goals checked. *This is a requirement, not a feature established at the audited pin.* At `Main.hs:5197-5211`, one path probes a search tactic, extracts replacement "Try this" text, and displays it after a goal-count decrease; it does not replay that replacement text. At `Main.hs:5265-5270`, another path constructs a final tactic combination using an `exact?` suggestion without probing that final spelling. A decrease in goal count can establish progress on a focused goal while other goals remain. These are static source findings, not reproduced runtime failures. The desired remedy is to replay the displayed candidate at the original context, distinguish progress from completion, and retain that result. A prior search state or success message is not the same receipt [16].

## 8.2 A concrete request and result contract

A synthesis request should contain a stable semantic header and separate search policy. The header includes the target type, current local context, fixed source expressions, behavioral predicate, chosen structures and indices, and the scope of allowed construction choices. The policy includes the finite registry, offered premises, method restrictions, ranking preferences, and budgets. A response should distinguish:

| Result | Evidence needed |
|---|---|
| Completed | Exact candidate plus proof of the frozen target, with no new premises. |
| Conditional route | Explicit residual telescope plus a proof under that telescope. |
| Candidate refutation | A proof of the negated source contract, possibly using a realized witness. |
| Model counterexample | A failing abstract observation whose source realization or negative-transfer theorem remains open. |
| Incomplete search | Sound discovered candidates or routes, with no exhaustive-failure claim. |
| No route in profile | Checked completion of the specified finite inference problem. |
| Unsupported request | The requested grammar, observation, or theorem is outside the service's contract. |

Do not collapse all of these into a Boolean "verified" field. A response also needs a target identity and registry version so that an accepted proof can be associated with the request that produced it. Hashes assist routing and cache integrity; Lean types and checked substitutions provide semantic authority.

### 8.3 One practical first integration

Start with exact candidate programs in the restricted list grammar and the free, empty, and diagonal domains. Reuse Leant's existing candidate and joint contract machinery. Add a theorem connecting each supported source constructor to its length interpretation, then use a frame certificate to produce an ordinary Lean length theorem for the exact candidate. Return both the candidate and that theorem in the response. Keep faster unchecked rankings available, but report their role accurately.

An informative failure is just as important: a model point outside the admissible image must remain a model counterexample, and a candidate outside the grammar must remain unsupported rather than receiving an invented affine law. This integration is narrow enough to test end to end and rich enough to exercise correlations and empty types that simple coefficient comparison misses.

## 9 Fresh execution results and adversarial review

### 9.1 Python reproduction

The three evidence lanes run unchanged Python sources in isolated package copies, with bytecode writing disabled and explicit UTF-8 I/O on Windows. They retain commands, input hashes, output hashes, logs, and exit codes. The documented suites and demonstrations meet their expected exits, including intentional rejection and capped-search cases. Source inputs under `docs/round-4/ideas` remain byte-identical to the pin.

Test units differ and should not be added into one misleading score. Examples of reproduced units include Alder's 4,096 configurations over its specified small unary-rule graphs; Clover's 35 regression tests and separate exact affine checks; Fennel's 27 boundary tests, 2,000 generated Horn graphs, 12,872 closure comparisons, and 1,820 certificate checks; Heather's 18 test methods; Juniper's 32 unit tests and 2,800 goal-frontier comparisons over 400 generated profiles; and Rowan's 32 tests with 12,675 list/model comparisons and 4,374 affine pair/image checks. The retained per-package outputs state the other units precisely. Counts establish the size of finite exercises, not a proof of the implementation or a human productivity result.

Across the lanes, all 36 documented suite/demo invocations meet their expected exit codes. An additional bounded comparison restricts Fennel and Juniper to their common proposition-indexed fragment: 128 profiles and 768 target frontiers agree, with 89 selected Fennel route certificates and all

128 Juniper results checked. Removing an alternative from a Juniper result and falsely marking it complete is rejected; marking the valid subset incomplete is accepted. Heather's additional domain mutations reject stale diagonal evidence after changing the domain to free inputs and reject an odd-length witness for an even-length request. These are independent probes of the stated finite boundaries, not a general equivalence or compiler-correctness proof.

Regeneration can change line endings, timing fields, paths, or deliberately variable output without changing the imported source. Fresh parity records distinguish byte equality from normalized-text equality. Copied historical metadata, including a hard-coded "no Lean executable" label in an upstream test output, remains historical or script-authored text. The root Lean receipt below is the authority for this review's actual compiler executions.

### 9.2 Lean acceptance: original files first

All 20 supplied Lean files are attempted serially with Lean 4.32.0. The source copies are byte-identical to the input pin. Clover's core is compiled to an isolated module for its importing examples. Heather's Mathlib imports use existing dependency artifacts from the read-only ProveIt checkout, with manifest revisions and tracked dependency status recorded. No Lake command, dependency rebuild, external cache update, or external source edit is used. Acceptance under these existing artifacts is not an independent full audit of the imported libraries.

**Nineteen of the twenty original files compile unchanged.** The sole failure is Heather's `even_reverse_length.lean`: `simp only` closes the goal and the following `omega` reports "No goals to be solved." The free reverse/append, diagonal, empty-domain, and readiness files compile. The exact per-file results are:

| Package | Accepted files | Meaning of the accepted declarations |
|---|---|---|
| Alder | 2 of 2 | Six conditional propositional theorems. |
| Bryony | 1 of 1 | Two conditional propositional theorems. |
| Clover | 2 of 2 | Seven concrete core rules and three generated applications. |
| Fennel | 4 of 4 | Four conditional propositional theorems. |
| Heather | 4 of 5 | Three concrete list theorems and one readiness theorem. |
| Juniper | 1 of 1 | Two conditional propositional theorems. |
| Laurel | 4 of 4 | Seven conditional theorems; the cycle module has no theorem. |
| Rowan | None supplied | No Lean acceptance claim. |
| Sorrel | 1 of 1 | One conditional propositional theorem. |

Thus 22 accepted declarations are generic conditional implications and 14 are concrete mathematical declarations, with an additional original concrete declaration rejected. Clover's seven explicit axiom queries report no axioms. These queries do not audit all other declarations. A theorem parameter named $r$ proves nothing about a proposed mathematical rule until the actual rule proof is supplied.

The six follow-up files are separate from those original-source counts. All three Clover adversarial exports—a reserved keyword, a shadowed composition name, and a shadowed rule name—are rejected by Lean. The Sorrel selection specimen compiles, confirming that its defect concerns route selection rather than theorem truth. Heather's one-line repair replaces the unguarded final tactic by `all_goals omega`; its original theorem statement is preserved and the repaired file compiles.

The additional `FreySharper.lean` file also compiles. It proves four named helper theorems culminating in the $p \geq 2$ first-coefficient divisibility and rational-cast statements, includes two $p = 3$ positive/negative examples, and queries the four theorem axiom sets. The queries report only `propext`,

`Quot.sound`, and, for three of them, `Classical.choice`. This is fresh kernel-acceptance evidence for the sharper arithmetic interface; it is not a proof of an entire curve construction.

### 9.3 What the adversarial cases test

| Case | Observed boundary | Required implementation response |
|---|---|---|
| Clover identifiers | A checked plan can emit a keyword or shadow a core name. | Generate hygienic identifiers and qualified expressions; compile exact output. |
| Heather tactic sequencing | A model-valid equality can be followed by a tactic after its goal is closed. | Use a robust proof-producing sequence; retain original and repair results separately. |
| Sorrel selection | Empty residual is known, but CLI chooses a route with new premise $A$. | Rank residual work and retain the matching derivation. |
| Rowan cyclic frontier | Offered diagnostic leaves do not suffice to close the goal. | Separate diagnostic explanations from sufficient contracts. |
| Partial antichains | A sound displayed support later becomes dominated. | Distinguish local normalization from global minimality. |
| Clover method root | A named root rule can be used redundantly. | Describe occurrence policy without claiming necessity. |

These probes do not invalidate the underlying mathematical designs. They demonstrate why isolated component tests are insufficient to validate the composed authoring system. In particular, rejecting malformed emitted Lean protects the kernel while still leaving a frontend reliability problem.

The output-directory probe is especially important: the current negative or no-route metadata is accurate, while old positive `.lean` bytes survive beside it. Compiling every file returned by a directory glob could therefore publish a true theorem from the wrong request. Use a fresh output directory or an authoritative manifest that binds each file to the request, its status, exact statement, and hash. Commit that manifest and its artifacts atomically.

Partial publication itself is a legitimate design choice. Alder refuses new output when any required `show` fails; Clover exports completed theorem nodes while marking the document incomplete. Distinguish saved partial content, individually proved assertions, and a completely checked document. Every displayed assertion, including one unused by the final theorem, must be accounted for before the whole document is presented as verified.

## 10 A combined implementation with explicit acceptance gates

### 10.1 Combine complementary components, keep one authority path

The recommended architecture has a small assertion layer over Lean. It fixes the target and structures; obtains typed rule instances from a bounded registry; uses a planner to find proofs or conditional routes; independently checks the selected plan; constructs ordinary Lean expressions; and checks the result against the original target. The readable proof is a projection of checked assertion nodes, so that expansion reveals the same argument rather than a separately authored explanation.

Reuse Clover's concrete term/rule discipline, one of the six antichain engines' support semantics, Juniper's coverage separation, Heather's behavioral domain fixtures, Rowan's sealed requests, and Sorrel's ordered residual interface. This is a recommendation to share contracts and fixtures, not to

merge six Python implementations into one trusted compiler. The planner may remain untrusted if its selected proofs are reconstructed and checked.

### 10.2 Five gates before a broad syntax experiment

1. **Exact mathematical registration.** Implement one real use site for quotient transport and one for endomorphism properties. Every generated rule instance carries a Lean theorem application; no opaque mathematical atoms masquerade as registrations.
2. **Completion fidelity.** The closed result has exactly the requested type. Conditional explanations remain visibly conditional. Keyword, shadowing, already-closed-goal, stale-context, and route-selection mutations are covered.
3. **Bounded inference.** Account separately for term generation, rule instances, support combinations, retained alternatives, and proof size. Distinguish finding one proof from certifying the entire support inventory.
4. **Behavioral bridge.** Complete one Leant candidate-to-length-proof path with free, empty, and correlated domains, plus realized negative cases.
5. **Authoring and repair measurement.** Compare equal mathematical libraries and services. Require users to recover hidden premises and diagnose mutated examples, not merely produce shorter source.

The first gate is intentionally mathematical. More proof-plan infrastructure without a real registration would repeat the boundary already explored by the six opaque-proposition engines.

### 10.3 A further synthesis: request-relative certificates

The failures suggest a stronger result interface than "a checked proof and some metadata." A completed response should be indexed by the original request and current context. A conditional response should additionally be indexed by its residual telescope and a checked substitution for already available evidence. A coverage response should be indexed by the finite profile and policy. A behavior response should name the exact source candidate and domain bridge. These indices describe what the evidence means; they are not only logging fields.

This leads to a useful composition rule: every stage must preserve or explicitly refine the request, and the final stage checks that refinement. The Sorrel selection probe violates this rule operationally even though its exported theorem is true. The Clover export probes violate it by failing to produce the promised Lean term. A single final closed-proof bit cannot explain either failure well enough for a mathematician to repair the document.

### 10.4 A further synthesis: separate the proof frontier from the work menu

An antichain answers a formal sufficiency question. The author needs a choice among meaningful next actions. Those actions may have different costs: prove parity, invoke a cached exactness lemma, expose a missing locality hypothesis, or choose a witness. A UI can group routes by mathematical action, hide dominated details, and rank likely work, while retaining the complete formal support behind each displayed action.

The ranking must not change truth or silently strengthen the theorem. A "one extra premise" route can be much harder than a two-premise route whose facts are already nearby. The naturalness experiment should therefore measure time to understand and discharge a selected route, not support cardinality alone. It should also ask whether a user can tell that an explanation is incomplete. This is where algorithmic frontiers become a language-design question rather than only a graph problem.

### 10.5 A further synthesis: domain certificates can be reused by many candidates

The affine-frame and semilinear results suggest caching the proof of an observation domain independently from each candidate's denotation proof. For a fixed typed input interface, one proved diagonal or empty-domain certificate can support many candidate comparisons. Candidate changes then rebuild the source-denotation bridge without re-proving the domain theorem. Conversely, a change in the input type, guard, or correlation invalidates the domain certificate even if the candidate's syntax is unchanged.

This separation gives a concrete incremental boundary and a useful cost model: domain construction, candidate reification, finite checking, and proof reconstruction should be timed separately. It also makes the asymmetry of acceptance and refutation visible: a negative test needs a realized input, whereas positive reasoning may use a sound span overapproximation.

## 11 Kernel changes and a fair experiment

### 11.1 The kernel question remains real, but is not yet the bottleneck

The reports consider a native refinement former with explicit evidence, law-bearing arrows, and changes to definitional equality. These deserve separate evaluation. An explicit refinement $\{x : A \mid P(x)\}$ can translate to an existing subtype or equivalent package. A primitive might improve representation or elaboration costs, but needs precise formation, introduction, elimination, erasure, and conversion rules together with metatheory.

Semantic subtyping or equality reflection is a stronger change: it risks putting proof search or external mathematical equivalence into conversion, whose predictability is crucial to type checking. The round-four prototypes' observed failures are mostly at generation, export, selection, and interface boundaries. None demonstrates that the existing kernel cannot support the intended mathematics. Implement the elaborator-level path first, and reopen the kernel experiment only for a measured limitation with a specified conservative interpretation or a deliberately justified new logic.

### 11.2 Matched controls

A study should distinguish the benefits of new mathematical lemmas, automation, presentation, and source syntax. Use these matched arms:

| Arm | What the participant receives |
|---|---|
| L0 | Current Lean and the existing relevant library. |
| L1 | Lean plus the new mathematical interface lemmas. |
| L2 | L1 plus exactly the automation, registry, solver, and synthesis services used by the proposal. |
| L3 | L2 plus the property-use and conditional-explanation interface. |
| L4 | L3 plus the proposed mathematical surface syntax. |
| Renderer | A readable projection of the same checked proof spine, evaluated separately from new source syntax. |

Randomize task order and use matched tasks rather than asking each participant to prove the same unfamiliar theorem repeatedly. Record experience with Lean and the mathematical domain. Use a pilot to calibrate tasks, keep held-out authoring tasks separate from the development examples, and charge the cost of new registrations, reifiers, provider proofs, and maintenance to the appropriate arm. A successful library or editor interface is an acceptable outcome; it does not need to be rebranded as a new language.

### 11.3 Measure omitted work and semantic recovery

Primary measures should include time to a correct complete proof, human interventions, recovery after a small edit, and ability to state the theorem's actual hypotheses. Record implementation costs separately: grounding volume, frontier width, time to first proof, time to complete inventory, proof term size, elaboration time, and incremental reuse failures. Source length is a secondary measure; fewer tokens can conceal a harder search or a weaker claim.

The paired mutations should include:

- Neighborhood equality replaced by point equality; a fixed measure replaced by another measure; countable a.e. interchange replaced by uncountable.
- Source divisibility removed; target cast nonzeroness removed; irrelevant Frey premises omitted while the sufficient local route remains intact.
- Independent lengths replaced by a diagonal domain or `List Empty`; an abstract witness outside the image; a candidate outside the proved grammar.
- A local assumption moved out of scope; a dependent witness or instance changed; an explicit method requirement applied after support minimization.
- A goal already supported by two facts competing with a smaller conditional route; a seedless cycle plus a leaf; an interrupted support inventory later dominated by an empty-support proof.
- An exported identifier equal to a keyword or registry declaration; a tactic sequence whose first tactic already closes the goal.

Each negative has a nearby positive, so an implementation cannot succeed simply by rejecting all requests in a difficult family.

### 11.4 Predeclared interpretations of results

If L1 captures most of the improvement, invest in mathematical APIs. If L2 does, invest in automation and discovery. If L3 improves premise recovery and repair beyond L2, the property interface has earned further development. If L4 adds measurable authoring or comprehension benefits beyond L3, a separate surface becomes justified. If only the renderer helps, improve proof presentation without claiming a new elaboration discipline is needed.

If exact registration, faithful export, or transparent residual status cannot be made reliable in the narrow slice, defer broad syntax and dependent generalization. A kernel extension should not be used to bypass those engineering gates.

## 12 Questions for the next iteration

The next responses should include code or experiments that answer these questions, rather than another broad catalogue of mathematical syntax.

1. **Which single actual Lean use site will be completed end to end?** Choose exact quotient transport or the concrete endomorphism registry, show the original target, and produce a closed Lean proof from parsed source with the emitted artifact replayed.
2. **What is the authority of a conditional route?** Specify its ordered residual telescope, how current facts discharge members, and the transition that permits publication. Include the Sorrel selection mutation.
3. **Which completeness claim is worth paying for?** Separate finding one proof, enumerating minimal supports, validating coverage, and generating all bounded instances. Report each cost and an honest budgeted result.
4. **How does grounding interact with alternatives?** Connect one typed term pool to an antichain engine. Show bounds, registration checks, dependent arguments rejected or handled, and a case where relevance filtering changes cost without changing the completed answer.
5. **What does a required method mean operationally?** Choose root occurrence, allowed support, a checked proof transformation, or another explicit policy. Include redundant use and same-support different-method examples; do not imply indispensability unless separately established.
6. **Which observation-domain theorem is reusable?** Implement free, empty, and correlated list domains with exact source bridges. Decide whether a frame suffices or an exact semilinear image is needed for the chosen contract.
7. **What does Leant retain after acceptance?** Demonstrate an exact candidate, its original behavioral target, a proof or replayable evidence, and a realized negative case using the committed baseline's existing machinery.
8. **How do edits invalidate evidence?** Change a local assumption, universe/index, or algebra instance and show correct rebuild or checked transport. Explain which cache entries survive and why.
9. **Can mathematicians distinguish an explanation from a theorem?** Test comprehension of conditional, incomplete, unsupported, and refuted results, including a cyclic diagnostic that is not a sufficient frontier.

10. **What evidence would justify a new surface or kernel feature?** Predeclare a measurable advantage beyond equally equipped Lean, identify the smallest feature responsible, and accept a library or renderer outcome when the experiment supports it.

# A Artifact guide and reproduction boundaries

The synthesis directory contains the TeX source and rebuilt PDF, a source register, critical reading memos, convergence records, isolated Python copies and logs, original Lean specimens and logs, adversarial specimens, and validation receipts. Source paths in the registers are repository-relative; the recorded absolute execution paths describe this Windows run.

Run `python -B scripts/verify_sources.py` to check the pinned corpus. The lane runners reproduce their finite experiments in their own evidence directories. Run `python -B scripts/check_lean.py` for the serial original Lean campaign after ensuring no other Lean build is running. Its existing Mathlib dependency paths are machine-specific and must be supplied on another host. It deliberately records original compiler failures rather than repairing the imported specimens. The companion follow-up runner identifies every additional source and expected result separately.

Run `./build.ps1` from PowerShell to compile this report in three strict pdfLaTeX passes without shell escape. Run `python -B scripts/render_review.py` to render every PDF page and produce numbered contact sheets. The final verification checks source and artifact hashes, compiler and reproduction receipts, PDF text and page inventory, build diagnostics, and the recorded visual review. The images are intermediate review artifacts; the PDF and its visual-review receipt are committed.

This report proposes a combined implementation. It does not modify the nine submitted packages or either external repository. Its retained counterexamples are review artifacts, not fixes silently applied to the original proposals.

# References

[1] **Alder.** *Residual Types and Proof-Producing Mathematical Elaboration.* Round 4, 2026. Report; source. See residual inference, realizable affine frames, and the executed Alder-0 sections.

[2] **Bryony.** Round-4 proof-language proposal, 2026. Report; source. See requirement families, grounding, weighted Fubini, and executable reference model.

[3] **Clover.** Round-4 proof-language proposal, 2026. Report; source. See typed grounding, endomorphism companion, profile comparison, and method policies.

[4] **Fennel.** *A Proof Language of Stable Objects and Explicit Requirements.* Round 4, 2026. Report; source.

[5] **Heather.** *Proof-Carrying Mathematical Interfaces.* Round 4, 2026. Report; source.

[6] **Juniper.** *Property-Directed Mathematics with Explicit Obligation Frontiers.* Round 4, 2026. Report; source.

[7] **Laurel.** Round-4 proof-language proposal, 2026. Report; source. See finite frontiers, partial-result qualifications, and sharp-regularity interfaces.

[8] **Rowan.** Round-4 proof-language proposal, 2026. Report; source. See sealed requests, bounded term generation, explanatory frontier, and semilinear images.

[9] **Sorrel.** Round-4 proof-language proposal, 2026. Report; source. See residual plans, finite support calculus, and Cauchy reconstruction.

[10] Codex. *From Properties to Proof Obligations.* Revised round-3 synthesis. Report. Consulted as secondary context; necessary definitions and corrections are restated here.

[11] *Nine Refinements.* Reconciled round-3 unified report at the current input pin. Report. Secondary context, not a tenth round-four proposal.

[12] Lean Language Reference. *Elaboration and Compilation.* https://lean-lang.org/doc/reference/latest/Elaboration-and-Compilation/. Consulted 6 September 2026; compiler executions in this review use the separately pinned 4.32.0 toolchain.

[13] Lean Language Reference. *Function Application.* https://lean-lang.org/doc/reference/latest/Terms/Function-Application/. Implicit, instance, and automatic arguments; consulted 6 September 2026.

[14] T. J. Green, G. Karvounarakis, and V. Tannen. *Provenance Semirings.* PODS 2007. https://repository.upenn.edu/bitstreams/b598c0a7-0d24-4162-8279-5f51a17d29c2/download. Related algebraic provenance background; not a claim that the proposed Lean frontend is already supplied by that work.

[15] Vladimir Reshetnikov and contributors. *Leant*, committed snapshot `823259f`. Candidate verification source. The preserved review records additional behavioral, length-contract, and handoff source boundaries; no fresh Leant runtime test is claimed.

[16] Vladimir Reshetnikov and contributors. *Leant Main.hs*, immutable `823259f` snapshot. Source. Candidate acceptance and tactic replacement paths are documented with exact ranges in `evidence/leant-review/source-register.json`.